Computer Communications 236 (2025) 108125



Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom





IoT edge network interoperability

Tanzima Azad [a,*], M.A. Hakim Newton [b], Jarrod Trevathan [a], Abdul Sattar [a]

[a] School of Information and Communication Technology, Griffith University, 170 Kessels Road, Nathan, 4111, QLD, Australia
[b] School of Information and Physical Sciences, The University of Newcastle, University Drive, Callaghan, 2308, NSW, Australia

ARTICLE INFO

Keywords:
Internet of Things
Edge devices
Network interoperability
Syntactic interoperability
Semantic interoperability

ABSTRACT

Network interoperability is crucial for achieving seamless communication across Internet of Things (IoT) environments. IoT comprises heterogeneous devices and systems supporting diverse technologies, protocols, and manufacturers. Enabling devices to communicate and exchange data effectively, regardless of underlying protocols, is key to building cohesive and integrated IoT networks. IoT has transformed multiple sectors ranging from home automation to healthcare—by harnessing a vast array of sensors and actuators that communicate through cloud, fog, and edge layers. However, the variety in device manufacturing and communication standards demands interoperable interfaces, and most current solutions depend on cloud-based centralised architectures. These architectures introduce latency and scalability challenges, particularly for resource-constrained IoT devices that often struggle to communicate with the cloud due to limited resources. This paper addresses network interoperability at the IoT edge level, focusing on resource-efficient communication by integrating Wi-Fi and Bluetooth, two commonly used protocols in IoT ecosystems. We have implemented a network edge interoperability solution that supports effective data exchange between devices operating on these distinct protocols, enhancing the overall efficiency, flexibility, and scalability of IoT systems. Our approach allows devices interoperate by addressing network latency and bandwidth limitations, incorporating an integrated controller to facilitate broader applications and enhance performance across IoT networks. Our findings illustrate how bridging protocol differences can foster more resilient and adaptable IoT solutions, advancing the deployment of IoT applications across various domains and use cases.

1. Introduction

The *Internet of Things* (IoT) is a network of connected devices that communicate and share data over the internet. These devices, ranging from household items to industrial machinery, use sensors and software for data exchange. IoT applications include smart homes with automated systems, healthcare with wearable monitoring devices, agriculture with optimised irrigation, and industrial settings where it enhances efficiency through predictive maintenance and real-time monitoring. However, the interoperability issues in IoT stem from the diversity of devices, communication technologies, and data formats, leading to challenges in seamless integration and data exchange. Effective solutions are needed to ensure smooth communication and compatibility across different IoT systems.

Interoperability in IoT is hindered by the wide array of communication protocols employed by devices, such as Wi-Fi, Bluetooth, Zigbee, LoRa, 6LoWPAN, and CoAP. While each protocol serves specific IoT use cases, no single protocol can address the entire spectrum of IoT requirements. This diversity creates compatibility barriers, limiting the ability of devices to operate cohesively within an unified IoT network. Current solutions often rely on cloud-based centralised architectures to address this issue. These architectures route data from heterogeneous devices through the cloud for processing and integration. Although this approach can facilitate interoperability in certain scenarios, it introduces significant latency and scalability issues, particularly for resource-constrained IoT devices with limited computational power and energy capacity. Such devices may struggle with cloud-based communication due to its bandwidth and energy demands, compromising the real-time responsiveness essential for applications like healthcare or industrial automation.

To address these limitations, achieving network-level interoperability in IoT edge devices is essential. This allows devices using different technologies and protocols to communicate and function together seamlessly, supporting smooth data exchange across diverse systems while using limited resources. By effectively integrating varied protocols, IoT networks can facilitate cohesive operations and advanced applications that depend on reliable, low-latency data sharing. Network interoperability at the edge becomes a foundation for enabling

* Corresponding author.
E-mail address: tanzima.azad@griffithuni.edu.au (T. Azad).

https://doi.org/10.1016/j.comcom.2025.108125
Received 26 November 2024; Received in revised form 13 February 2025; Accepted 27 February 2025
Available online 11 March 2025
0140-3664/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

diverse systems to work together, even with resource-constrained devices, thereby driving the development of robust and adaptable IoT applications and services.

This paper prioritises support for emergency and mission-critical services, which require ultra-reliable and low-latency communication [1,2]. Edge computing is ideally positioned to address these needs by placing computational resources closer to data sources and users. This proximity yields substantial benefits, including decreased latency, optimised bandwidth utilisation, and enhanced resiliency and availability. These advantages are particularly critical in unpredictable and resource-limited settings, such as those encountered by first responders, law enforcement, and military personnel, where rapid and informed decision-making can be life-saving.

Among various possible IoT applications, we present a use case highlighting network interoperability in a time-critical scenario. Imagine a drone equipped with a thermal camera detecting a heat signature, indicating a survivor trapped under rubble. The drone transmits this data using the Zigbee protocol to a ground robot, which operates on Wi-Fi. The robot then relays the critical information to the rescue team's wearable devices via Bluetooth. In such a time-critical scenario, network interoperability ensures successful and fast transmission of vital messages across different network protocols. Note that such a transmission is more crucial than handling large volumes of data or facilitating communication among numerous devices.

Nevertheless, a unified IoT system should support multiple network protocols, ensuring seamless communication across diverse connectivity standards. This requires a flexible approach that accommodates both existing and new protocols without altering the underlying infrastructure. Organisations such as the Open Connectivity Foundation (OCF), IP for Smart Objects (IPSO), and W3C are working to establish standard models for IoT devices. However, the different ways in which devices communicate using varied protocols, and technologies continues to be a significant challenge. Thus, network interoperability solutions that can bridge multiple protocols are essential.

To address network interoperability in IoT, various approaches are used. IP-based methods embed the TCP/IP stack into devices for direct communication, reducing the need for gateways [3]. Software-Defined Networking (SDN) translates different protocols into a common format, enhancing integration and management [4]. Network Function Virtualisation (NFV) allows multiple virtual networks to share infrastructure, lowering costs and increasing flexibility [5]. These strategies help overcome interoperability challenges, creating more cohesive IoT systems. Another work, the Routing Protocol for Low-Power and Lossy Networks (RPL) has a goal to achieve reliable connectivity in various network environments by optimising routing [6]. The Internet Engineering Task Force (IETF) has established standards for implementing IPv6 over low-power wireless area networks, known as 6LoWPAN [7]. This protocol is designed to optimise IPv6 communication specifically for the unique demands of low-power wireless networks.

In this paper, we have implemented a network interoperability solution following an IoT edge framework [8] named InterEdge. Our implementation allows communication between Bluetooth and Wi-Fi while keeping further options to accommodate other existing and even new protocols in future. We have implemented network profiles for Wi-Fi and Bluetooth. We then compared our network interoperability implementation with three open standards — OGC SOS, OGC SensorThings API, and EdgeX Foundry using ROM usage, RAM usage, and response latency as evaluation metrics.

Our experiment shows that SOS uses the most ROM, while SensorThings API and EdgeX require less. Our implementation for Bluetooth uses more ROM than the three standards but uses less RAM, offering better RAM efficiency. The Wi-Fi implementation in our solution is most efficient in both ROM and RAM usage, indicating improved performance. SOS has the highest response latency due to larger data sizes, while EdgeX achieves the fastest response times by reducing cloud interactions. Our implementation using Bluetooth and Wi-Fi outperform SOS and SensorThings API in response latency, though they remain slightly slower than EdgeX.

To summarise, the contributions of this paper are below.

- Addressing the implementation challenges to obtain an actual implementation of the network interoperability level of the theoretical interoperability model and framework InterEdge [8]. Unlike existing cloud-dependent solutions, which struggle with mission-critical and emergency services, this work focuses on enabling ultra-reliable, low-latency communication for time-sensitive applications.
- A comparison of our solution with three existing open standards for interoperability. Our approach features an integrated controller and explicit network-level interoperability. Our approach demonstrates superior performance than the other solutions in code size, memory utilisation, and response latency.

The rest of the paper is organised as follows: Section 2 defines the terminologies; Section 3 delves into existing solutions for IoT network interoperability; Section 4 gives an overview of the InterEdge interoperability framework [8]; Section 5 provides the implementation details of our network interoperability solution; Section 6 describes example edge network scenarios. Section 7 provides a comparative analysis of our proposed network interoperability implementation against three existing standards. Lastly, Section 8 presents our conclusions and suggests potential future research.

2. Terminologies

We adopt the terminologies used in the hierarchical decentralised interoperability framework, InterEdge [8], and briefly review them below. Additionally, we define two new terminologies, *framework* and *implementation*, to clearly differentiate the contributions of the two papers.

1. **Model:** A categorisation of interoperability: hierarchical or non-hierarchical, vertical or horizontal, or mixed.
2. **Level:** A category in a hierarchical or vertical interoperability model. A higher level with larger granularity sequentially depends on a lower one in the hierarchy.
3. **Class:** A category in a horizontal interoperability model. Classes do not mutually depend on each other.
4. **Framework:** A potentially implemented solution.
5. **Implementation:** An actually implemented solution.
6. **Solution:** An implementation framework that potentially implements an interoperability model. It could also mean an actual implementation of an interoperability model. We use solution when we do not need to explicitly distinguish between a framework or an implementation.
7. **Service:** An algorithmic method that implements a particular data or execution functionality for a solution.
8. **Layer:** A set of services sharing certain traits of functionalities within a vertical organisation of the services.
9. **Environment:** A connected network system of physical objects, devices, sensors, and actuators that interact.
10. **Interoperable Environment:** An environment that ensures seamless interoperability among its physical objects, devices, sensors, and actuators.
11. **Data and Message:** Data goes in or comes out of devices. Messages are data plus routing information and are circulated over the network.

Note that a potential implementation could just mention about certain attribute while an actual implementation would also specify whether plain text or XML or some format to be used for the attribute.

3. Related work

Network interoperability is key to seamless communication and data exchange in IoT. It allows diverse devices and systems to interact smoothly, regardless of their protocols, technologies, or manufacturers. This enables the creation of unified, integrated IoT networks, leading to more efficient and flexible IoT ecosystems.

3.1. Software defined networking (SDN)

Software defined networking (SDN) revolutionises IoT networks by enhancing intelligence, efficiency, security, and scalability through the separation of control and data planes [9]. Addressing key challenges like device heterogeneity, mobility, quality of service (QoS), and security, SDN fosters seamless communication among diverse IoT devices by standardising data formats [4,10]. Advanced algorithms, such as genetic algorithms, optimise resource allocation and flow scheduling, surpassing traditional methods. However, SDN's effectiveness is tempered by ongoing challenges in managing device heterogeneity and applications. While IPv6 addresses large-scale connections, diverse device characteristics remain a hurdle. Moreover, mobility management in IoT networks necessitates solutions beyond current protocols like PMIPv6 [11].

3.2. Network function virtualisation (NFV)

Network function virtualisation (NFV) synergies with SDN by decoupling network functions from physical hardware, empowering service providers to create virtual networks that leverage shared infrastructure. This optimises operational and capital expenditures through resource sharing and flexible function deployment. In the IoT realm, NFV integrates with SDN to form a layered SDN-IoT framework, where a network operating system (NOS) orchestrates SDN devices and IoT gateways. However, the lack of concrete guidelines for applying virtualisation to address the diverse requirements of IoT use cases remains a challenge [12].

3.3. Device communication methods

To provide a clear understanding of network interoperability, [13] explored the end-to-end communication process, from the embedded device to the final delivery of data to the end device or user. The three communication methods outlined in [14] are Device-to-Gateway, Device-to-Device, and Device-to-Cloud. Each method has specific use cases, with Device-to-Gateway being the most commonly employed for achieving interoperability in IoT. However, all three methods play important roles in different IoT scenarios. Due to the limited processing power and storage, most IoT devices are unable to connect directly to the cloud. This restricts the network capability of IoT devices by classifying them into distinct classes according to their capabilities [15,16].

3.4. Networking protocols

To enable networking interoperability in the IoT, various networking protocols and technologies have been employed. For instance, traditional protocols like Universal Plug and Play (UPnP) and Digital Living Network Alliance (DLNA) are used to connect between IoT devices and gateways. IP based approaches allow direct communication between sensor networks and IP networks, improving interoperability. However, resource constraints make them challenging, leading to solutions like 6LoWPAN and CoAP. These solutions enhance connectivity but introduce security risks [17]. A detailed analysis of the network stack, as discussed by the Internet Engineering Task Force (IETF), reveals that CoAP is incompatible with non-IP technologies such as Bluetooth and ZigBee [18].

3.5. IoT gateway

Like the approach of trying to solve network interoperability using IP based approach another approach is to build IoT gateway. Now, the paper in [19] has mentioned that the IoT gateway is a crucial component of IoT networks, similar to conventional gateways but with additional functions. It must support heterogeneous network connectivity, handling both short-range communication protocols (like ZigBee, Bluetooth, and Z-wave) and high-performance ones (such as Wi-Fi and LTE). Moreover, it has enhanced network manageability, enabling it to manage and update connected devices. The gateway must also support protocol and platform interworking to ensure compatibility between legacy devices and new IoT platforms, solving standardisation and compatibility challenges.

3.6. Interoperability between Wi-Fi and Bluetooth

In this paper, we demonstrate the interoperability of Wi-Fi and Bluetooth communication protocols through our interoperable solution. This capability is increasingly critical in diverse IoT environments where devices often need to communicate seamlessly across different wireless technologies to optimise connectivity, power consumption, and data transfer efficiency.

Bluetooth, standardised by the IEEE 802.15.1 [20,21] working group, is a short-range wireless technology designed for both data and voice transmission. Typically operating within a range of 10 m, Bluetooth's reach can extend up to 100 meters with higher transmission power. Bluetooth operates within a frequency range of 2.4 to 2.4835 GHz and typically supports transmission rates of up to 1 Mbps [22]. This makes it well-suited for applications that prioritise low data rates and minimal power consumption.

Wi-Fi, defined by the IEEE 802.11 standard [20,23], encompasses a suite of wireless local area network (WLAN) technologies that provide high-speed internet connectivity over longer ranges than Bluetooth. Operating primarily within the unlicensed 2.4 GHz and 5 GHz frequency bands (and more recently in the 6 GHz band with Wi-Fi 6E), Wi-Fi supports a wide range of transmission rates, from 54 Mbps in 802.11a/g to several Gbps in the latest standards such as 802.11ax (Wi-Fi 6). Wi-Fi 7 (802.11be) introduces faster speeds up to 46 Gbps with features like 320 MHz channels, 4 K-QAM modulation, and multi-link support [24]. Transmission power can vary but typically ranges between 50 mW and 1000 mW, depending on regional regulations and specific device configurations. Using spread spectrum and frequency-hopping techniques, Wi-Fi ensures reliable communication, even in crowded radio frequency environments. This unlicensed spectrum enables flexible deployment across diverse devices without regulatory constraints.

Achieving interoperability between Wi-Fi and Bluetooth presents significant challenges. To address this, we define network profiles for both Bluetooth and Wi-Fi that specify each technology's range, data rate, and frequency characteristics. These profiles allow a controller to interpret the networking parameters of Bluetooth and Wi-Fi, enabling it to establish communication with devices operating on either protocol. By facilitating a connection between devices across these two technologies, our solution enables seamless communication that adapts to application requirements, available bandwidth, and power constraints, thereby enhancing network flexibility and improving device interoperability.

3.7. Comparative analysis of IoT interoperability solutions: SOS, SensorThings API, and EdgeX Foundry

We subsequently empirically compare our implementation with three contemporary interoperability solutions: SOS [25], SensorThings API [26], and EdgeX Foundry [27]. SOS is exclusively cloud-based. SensorThings API and EdgeX Foundry support both cloud and edge-level

Table 1
Edge Interoperability Model [8]: the interoperability levels and their implementing profiles.

Interoperability level	Implementing profile
Organisational	Organisation Profile
Semantic	Device Profile
Syntactic	Data Profile
Network	Network Profile

interoperability. However, they still require cloud access to facilitate edge-level interoperability. We delve deeper into these three solutions.

The Sensor Observation Service (SOS) [25], developed by the Open Geospatial Consortium (OGC), is a standardised web service designed to address interoperability challenges in IoT. SOS allows users to integrate sensor data from various sources using a common format and supports sensor registry management. However, its cloud-based architecture requires a stable network connection, limiting its effectiveness in areas with poor internet connectivity. Unreliable networks can lead to delays or failures in data transmission and retrieval, and the large XML data requests can be too demanding for low-bandwidth networks. Additionally, SOS does not support tasking services for actuators or receivers.

The SensorThings API [26], a JSON-based alternative to XML for SOS, was introduced by OGC to address the overhead associated with XML, especially for resource-constrained devices. This API features two main profiles: the Sensing Profile, which organises observations into data streams linked to specific sensors and features, and the Tasking Profile, which facilitates the control and management of actuators by specifying tasks and actions. Despite its design for edge devices, the SensorThings API faces implementation challenges due to the limited processing power and memory of these devices, hindering its widespread adoption and integration across various IoT ecosystems.

The EdgeX Foundry [27] is an open-source framework designed to address interoperability issues in IoT edge devices. It acts as a translator, providing a common platform for connecting and managing devices and applications, which enables seamless communication and data exchange. However, its architecture depends on consistent network connectivity between edge devices and the central platform. Disruptions in internet connectivity can hinder data synchronisation between edge devices and cloud services, negatively affecting the system's overall reliability and performance.

4. Framework overview

We provide an overview of the hierarchical decentralised edge interoperability model and its associated framework [8].

4.1. Model and framework

Table 1 left column shows the interoperability levels in the model. Fig. 1 shows the layers in the associated implementation framework.

The interoperability model in Table 1 has four levels: *network*, *syntactic*, *semantic*, and *organisational*. The network level deals with networking technologies to establish seamless connections among edge devices. The syntactic level supports data formats used in communication. The semantic level provides the meaning of the messages transmitted. The organisational level addresses platform and domain heterogeneity.

In this paper, we focus on achieving network interoperability. We demonstrate interoperability between two widely used network protocols: Wi-Fi and Bluetooth. However, to evaluate the network level, we do need some implementations of the syntactic and the semantic levels although achieving these two levels is out of scope of this paper. To implement syntactic and semantic interoperability, we created a device profile for the DHT11 temperature sensor and a controller profile for the ESP32 microcontroller. We then leverage these profiles to achieve network-level interoperability using the DHT11 device profile and the ESP32 controller profile.

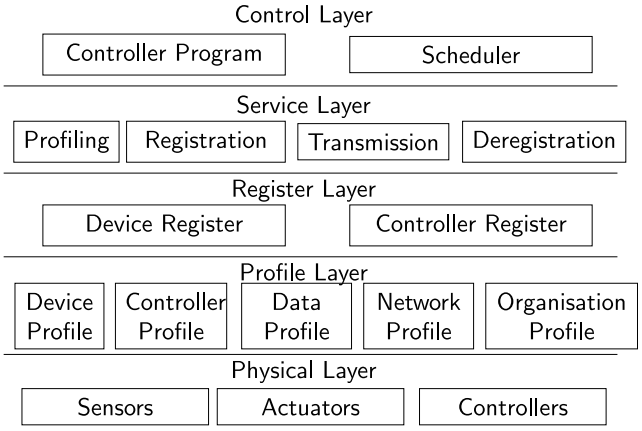


Fig. 1. Edge Interoperability Framework [8].

We chose to demonstrate interoperability between Wi-Fi and Bluetooth because they are the most widely used protocols in IoT edge devices. Wi-Fi is ideal for high-speed, data-intensive communication, while Bluetooth offers low-power, short-range connectivity. Ensuring seamless interoperability between these widely adopted technologies is crucial for enhancing the functionality and efficiency of IoT ecosystems. Furthermore, our framework is designed to be adaptable, making it possible to extend and implement network interoperability across other communication protocols as needed. In this study, we focus specifically on Wi-Fi and Bluetooth to showcase our framework's capabilities in achieving network-level interoperability for widely used IoT protocols.

The proposed implementation framework in Fig. 1 has five layers: *physical*, *profile*, *register*, *service*, and *control*. The physical layer holds the actual devices, while the profile layer defines the characteristics and capabilities of these devices. The register layer maintains a record of active devices and controllers. The service layer provides essential services: *profiling*, *registration*, *transmission* and *deregistration*. Finally, the control layer orchestrates the entire system by continuously monitoring devices, collecting data, and initiating necessary actions.

As shown in Table 1 right column, interoperability at various levels is achieved through specific profile parameters in our framework: network profile parameters for network level interoperability, data profile parameters for syntactic level interoperability, device profile parameters for semantic level interoperability, and organisation profile parameters for organisational level interoperability.

4.2. Program control flow

Before executing the program, key profiles (device, controller, data, network) defining devices, controllers, data, and network are stored in non-volatile memory, along with the edge environment's configuration in the device and controller registers. Upon running the program, the controller loads these profiles and registers into volatile memory before starting execution. It then operates in a continuous loop, using a scheduler to manage and execute tasks. In the next section, we provide a detailed discussion of each layer, the associated services, the functionality of the controller and scheduler, and the overall edge network architecture.

5. InterEdge network interoperability implementation

This paper demonstrates the practical application of the previously proposed theoretical framework, proving its real-world viability

and tackling crucial implementation challenges. A notable challenge is the resource constraints of edge devices, which are essential for time-sensitive applications that rely on immediate data processing and decision-making. While implementing syntactic and semantic levels, which are out of scope of this work, we observed that XML and JSON were impractical due to memory constraints, leading to the adoption of plain text files for reduced memory usage. Then the framework was implemented in C/C++ for smaller executable size, faster performance, and compatibility with microcontrollers like Arduino Uno R3 and ESP32. We stick with the same choices in this work.

1. The framework supports class-0 devices, but a class-1 device (ESP32) was used as the controller to implement network interoperability. This is because the controller needs to at least support the network interoperability protocol. Different network protocols operate on varying bandwidths and latencies. So network profiles were implemented as such. These allow the controller to interpret messages based on the sender's protocol and relay them to the receiver using its respective protocol, ensuring seamless communication across diverse IoT devices.
2. Developing network profiles required a deep understanding of each network protocol, including its data transmission and reception mechanisms. Ensuring network interoperability demands comprehensive knowledge of protocol functionalities and their effective utilisation for seamless communication.

Below we describe the details of each layer of our implementation. In this implementation, a Bluetooth terminal is connected to the first controller. The terminal transmits either an 'ON' or 'OFF' message to the first controller via Bluetooth. Upon receiving the message, the first controller relays it to a second controller through a wired connection. The second controller, in turn, communicates with a web server hosted by a third controller via Wi-Fi. Based on the received message from the web server via Wi-Fi, if the third controller receives an 'ON' command, it turns on the connected LED; conversely, if it receives an 'OFF' command, it turns off the LED. While we demonstrate the implementation for two widely used network protocols, it is possible to extended to support additional network protocols in the future.

5.1. Physical layer

The physical layer encompasses all the physical devices. We need controller, sensor, and actuator devices. Below we list the specific hardware used as these devices.

1. **Sender Devices:** Bluetooth Terminal
2. **Controller Devices:** ESP32 microcontroller
3. **Actuator Devices:** Light Emitting Diode (LED) light

The controller devices are connected with each other through Bluetooth, Wired, and Wi-Fi. The sender is connected with a controller via Bluetooth and the actuator is connected with controller using wired connection.

5.2. Profile layer

The profile layer stores detailed information about all the devices in the edge network. The information includes device type, manufacturer, configuration parameters, network protocols, data formats, and organisational affiliations. These profiles include main parameters and further parameters for reading and writing data.

Table 2
Device profile for Bluetooth Terminal.

Parameter	Value
TypeID	Type004
TypeName	Bluetooth Terminal
Category	Communication
SubCategory	Terminal
TransmissionMode	Send
Connectivity	Bluetooth
DataType	Text
Encoding	UTF-8
NeedFormula	No

Table 3
Data profile for plain text.

Parameter	Value
Id	Format002
Name	Plain Text Format
Version	PT01
DataType	Text
Encoding	UTF-8

5.2.1. Device profile

We have used device profiles for We have created device profile for Bluetooth Terminal in Table 2.

- **Bluetooth Terminal:** Table 2 shows the parameter values for the device profile.
- **LED Light:** We used the parameter values for the LED light device profile which is The LED light device, identified as Type003, belongs to the sensor category and the light subcategory. It uses a wired connection with a write transmission mode. The data type and measuring unit are Boolean, and it does not require a formula for operation. The write method used is `WriteData`. The `WriteData` method for its parameter values is divided into two main sections: Initialisation and Write operations. The key parameters for the `WriteData` function include noOfPin, pins, and pinMode for initialisation, while the Write function specifically requires pinMode. In the Initialisation phase, the number of required pins must be specified, with the designated pin assigned to the LED and its mode set to OUTPUT. During the Write phase, the pin mode remains OUTPUT, ensuring the LED functions correctly.

5.2.2. Controller profile

A controller profile provides essential information in a structured format about a microcontroller, including its main and configuration parameters. This standardised profile supports consistent setup, integration, and interoperability across IoT environment.

- **ESP32 Microcontroller:** The controller profile for the ESP32, identified by TypeID (ContType002) and TypeName (ESP32). It belongs to the category of Xtensa microcontrollers and supports various connectivity options, including wired connections, Wi-Fi, and Bluetooth. It uses the network profiles for Wi-Fi, and Bluetooth that we have created in this paper.

5.2.3. Data profile

We have created data profile for plain text format in Table 3 to send plaint text format data from one controller to another controller. The data profile is associated with a device when the device is registered to the network.

- **Plain Text:** Table 3 shows the parameter values for the data profile.

Table 4
Network profile for Bluetooth.

Parameter	Value
Id	Network001
Name	Bluetooth
Libraries	BluetoothSerial.h
Bluetooth Name	ESP32-A
Functions	Serial.begin SerialBT.begin SerialBT.available SerialBT.read Serial.write

Table 5
Network Profile for Wi-Fi.

Parameter	Value
Id	Network002
Name	Wi-Fi
Libraries	Wi-Fi.h HTTPClient.h WebServer.h
ssid password	Wi-Fi-82C65 11973331
Status (connected) Status (Disconnected)	WL_CONNECTED WL_DISCONNECTED
Functions	Wi-Fi.begin Wi-Fi.status Wi-Fi.localIP HTTPClient::begin HTTPClient::addHeader HTTPClient::POST HTTPClient::getString WebServer::begin WebServer::on WebServer::send

- **Digital Signal:** The data profile for a digital signal is identified by the parameter Id (Format001) and Name (Digital Signal), with version DS01. The data type is specified as "Digital Signal", and no encoding is required.

5.2.4. Network profile

We create network profiles for both Bluetooth and Wi-Fi connections. These profiles encompass the essential parameters required for establishing and managing connections over Bluetooth and Wi-Fi networks, as well as facilitating data exchange between devices.

- **Bluetooth**
  Table 4 shows the parameter values for the Bluetooth network profile.
- **Wi-Fi:**
  Table 5 shows the parameter values for the Wi-Fi network profile.

5.2.5. Organisation profile

In this work, organisational-level interoperability was not implemented, and thus, no organisation profile was created. For network-level interoperability, only a single type of microcontroller (ESP32) was utilised alongside basic raw sensors (DHT11) and actuators (LEDs), all of which communicate via digital signals and plain text. The creation of organisational profiles would have been necessary if devices, such as microcontrollers, sensors, and actuators from different manufacturers or applications from different domains, had been integrated.

Table 6
Device register for Bluetooth Terminal.

Parameter	Value
DeviceId DeviceName	BTTERMINAL001 Bluetooth Terminal
TransmissionMode	Send Text Message
Connectivity DeviceProfile DataProfile NetworkProfile OrganisationProfile	Bluetooth Type004 Format002 Netowrk001 N/A
ActualValue ConvertedValue	N/A N/A

Table 7
Device register of LED Light.

Parameter	Value
DeviceId DeviceName	LED001 LED Light
TransmissionMode	Write
Connectivity DeviceProfile DataProfile NetworkProfile OrganisationProfile	Wired Type003 Format001 N/A N/A
ActualValue ConvertedValue	N/A N/A

5.3. Register layer

The register layer serves as a repository for managing active devices and controllers within the IoT network. It is important to note that some registration fields overlap with those found in device and controller profiles. This redundancy arises from the distinction between profiles, which typically outline a range of potential configurations, and the registration layer, which reflects only the specific subset of configurations actively employed in the network. This overlap ensures flexibility, allowing the system to adapt dynamically to various scenarios by selecting and applying the most relevant configurations.

Since registration involves an edge environment, we show an example in Fig. 2. The example has controller to device connections as well as controller to controller connections.

5.3.1. Device register

For each instance of a device type, we created a unique device register and assign a distinct DeviceId. For the Bluetooth Terminal which is actually the sender and the actuator device which is the LED in Fig. 2, we will create two device registers: one for the Bluetooth Terminal, and one for the LED light.

- **Bluetooth Terminal:** Table 6 shows the parameter values of the device register for Bluetooth Terminal.
- **LED Light:** We have one LED light in Fig. 2. So we need one device register for the LED light. Table 7 shows the parameter values for the device register for the LED.

5.3.2. Controller register

We create three controller registers for the three ESP32 microcontrollers. Each controller register gets its unique ContollerId.

- **ESP32 Controllers:** Table 8, Tables 9 and 10. show the parameter values for three controller registers. Notice that each controller keeps the other controller in its ConnectedControllers field.

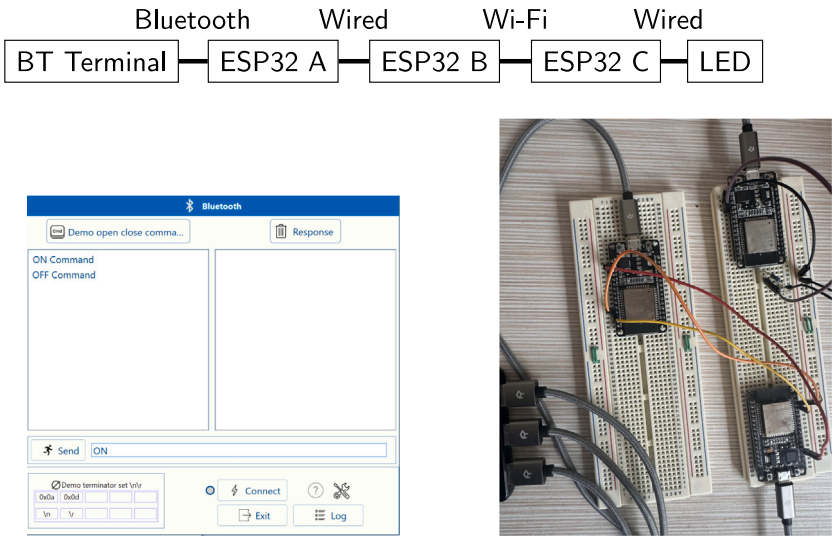


Fig. 2. An example edge environment with three controllers (ESP32), Bluetooth Terminal (as sender), and one actuator (LED). All five devices to be registered. Top: schematic diagram and Bottom: real experimental image with the Bluetooth Terminal.

Table 8
Controller register for ESP32 A.

Parameter	Value
ControllerId	Esp3200A
ControllerName	ESP32
ConnectedDevices	Esp3200A-BTTERMINAL001
ConnectedControllers	Esp3200B
Connectivity	Bluetooth, Wired
NetworkProfile	Network001
OrganisationProfile	N/A
SourceDevices	Esp3200A-BTTERMINAL001
ConditionActions	N/A
DestinationDevices	ESP3200A-ESP3200B-ESP3200C-LED001
DeviceProbeIntervals	(00:00:05)
MessageBuffer	NULL
ControllerProbeIntervals	(00:00:05)

Table 9
Controller register for ESP32 B.

Parameter	Value
ControllerId	Esp3200B
ControllerName	ESP32
ConnectedDevices	N/A
ConnectedControllers	Esp3200A, Esp3200C
Connectivity	Wired, Wi-Fi
NetworkProfile	Network002
OrganisationProfile	N/A
SourceDevices	N/A
ConditionActions	N/A
DestinationDevices	Esp3200B-Esp3200C-LED001
DeviceProbeIntervals	(00:00:05)
MessageBuffer	NULL
ControllerProbeIntervals	(00:00:05)

Table 10
Controller register for ESP32 C.

Parameter	Value
ControllerId	Esp3200C
ControllerName	ESP32
ConnectedDevices	Esp3200C-LED001
ConnectedControllers	Esp3200B
Connectivity	Wi-Fi
NetworkProfile	Network002
OrganisationProfile	N/A
SourceDevices	N/A
ConditionActions	N/A
DestinationDevices	Esp3200C-LED001
DeviceProbeIntervals	(00:00:05)
MessageBuffer	NULL
ControllerProbeIntervals	(00:00:05)

5.4. Service layer

The service layer encompasses four types of services: profiling, registration, transmission, and deregistration.

1. **Profiling:** Profiles for devices, controllers, data, and network are created and stored in the non-volatile memory of respective microcontrollers.
2. **Registration:** Registers for devices and controllers are created and stored in non-volatile memory, although the framework [8] supports keeping registration data in the random access memory.
3. **Transmission:** Seven methods manage data transmission and We have implemented all the methods as their algorithms are in paper [8].
   - ReadData connects to devices, reads, and processes data.
   - NeedAction uses artificial intelligence algorithms to determine actions.
   - WriteData writes processed data to devices.
   - NeedRouting decides if data routing is necessary.
   - WriteMessage prepares messages for network transmission.
   - RouteMessage transmits messages between controllers.
   - ReadMessage reads incoming messages from other controllers.
4. **Deregistration:** Device information is removed from the controller register when no longer needed, with updates stored in non-volatile memory. The framework [8] also supports storing registration data in the random access memory.

5.5. Control layer

The control layer features a ControllerProgram and scheduling methods such as InitSchedule, UpdateSchedule, DataReadingScheduled, and MessageReadingScheduled to ensure continuous data and message retrieval. The controller uses scheduled intervals and methods

such as ReadData, NeedAction, NeedRouting, WriteData, WriteMessage, RouteMessage to read data, determine actions, and manage routing. Data is written directly to devices or routed through controllers, with regular checks of the MessageBuffer for new messages. The ControllerProgram runs in a loop, updating schedules and performing actions in sync with the devices. Asynchronous scheduling is supported by using a synchronously probed switch to trigger immediate or scheduled actions.

6. Implementing edge networks

6.1. Generic edge networks

Network interoperability is achieved through network profiles tailored for different communication protocols. In this implementation, profiles for Wi-Fi and Bluetooth have been created, allowing seamless use across various devices and controllers. However, integrating a new network protocol would require developing a corresponding profile for that protocol.

In designing an edge network, device, controller, data, and network profiles are stored in the controller's EEPROM. The controllers are then registered in the network via controller registers, followed by the sequential registration of devices through device registers. When a controller reaches its capacity, an additional controller is introduced, and new devices are registered accordingly. In an edge network, device communication relies on active connections. While wired connections naturally maintain continuous communication, wireless protocols like Wi-Fi and Bluetooth require persistent connectivity to enable seamless data exchange between devices.

6.2. Example edge network

The ESP32 microcontroller, with its limited processing power and memory (typically 520 KB of SRAM and up to 4 MB of flash), can only support one communication protocol at a time. Consequently, applications that require both Bluetooth and Wi-Fi connectivity need separate ESP32 microcontrollers for each protocol. In our edge network example, we used separate ESP32 devices for Bluetooth and Wi-Fi. Specifically, one ESP32 handles Bluetooth communication, while another manages Wi-Fi. To test this setup, we configured four ESP32 microcontrollers, a DHT11 temperature sensor, and an LED light. In this scenario, the LED is remotely controlled based on temperature readings from the DHT11 sensor, with commands sent by the first controller and relayed across the ESP32 devices. This setup successfully demonstrates interoperability between Bluetooth and Wi-Fi, enabling seamless command transmission across protocols to control the LED remotely. The communication flow is given below (see Fig. 3).

6.2.1. Temperature monitoring and bluetooth communication

The system begins with the first ESP32 microcontroller (ESP32-A), which is connected to a DHT11 temperature sensor. ESP32-A continuously monitors the temperature and compares it to a predefined threshold. If the temperature exceeds this threshold, ESP32-A sends an "ON" message via Bluetooth to the second ESP32 microcontroller (ESP32-B). The `BluetoothSerial` library is used to establish Bluetooth communication, with ESP32-A configured to transmit data when the threshold is surpassed. This stage illustrates the integration of sensor data with Bluetooth communication, enabling wireless transmission of temperature-based triggers.

6.2.2. Serial communication and message forwarding

The second ESP32 microcontroller (ESP32-B) is responsible for receiving the message from ESP32-A over Bluetooth and forwarding it to the third ESP32 (ESP32-C) via a wired serial connection. ESP32-B operates as an intermediary, bridging Bluetooth communication with wired serial communication. Upon receiving the "ON" message, ESP32-B transmits the data to ESP32-C through UART, showcasing interoperability between different communication methods and the ability to relay commands efficiently within the network.

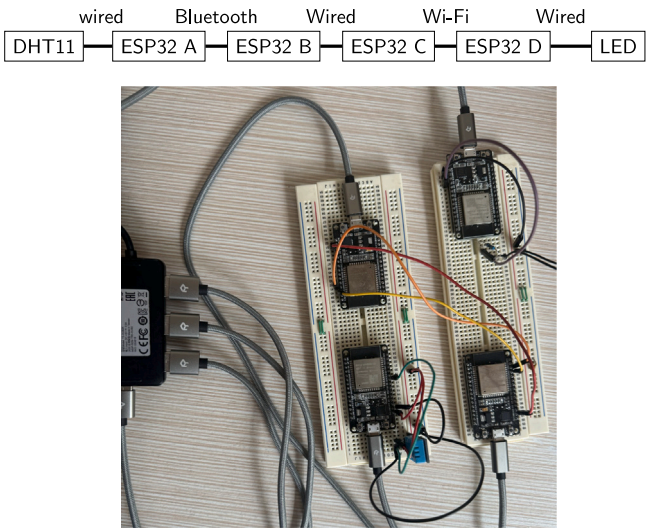


Fig. 3. Multiple Controller Scenario: Four ESP32 microcontrollers are connected with one DHT11 temperature sensor and one LED. Top: schematic diagram and Bottom: real experimental image.

6.2.3. Wi-Fi communication and web server request

ESP32-C, the third ESP32 in the system, is equipped with Wi-Fi capabilities and connected to the network using the `Wi-Fi` library. Upon receiving the message from ESP32-B, ESP32-C constructs an HTTP POST request and sends it to a web server hosted by the fourth ESP32 (ESP32-D). This step highlights the seamless transition from wired serial communication to Wi-Fi-based data transmission, demonstrating the flexibility of integrating multiple communication protocols in IoT networks.

6.2.4. Web server and LED control

The fourth ESP32 microcontroller (ESP32-D) operates as a web server, utilising the `WebServer` library to handle incoming HTTP requests. It also controls an LED connected to its GPIO pins. When ESP32-D receives an "ON" message via an HTTP POST request from ESP32-C, it activates the LED. Conversely, if an "OFF" message is received, the LED is turned off. The `generateHtmlResponse()` function dynamically generates an HTML page reflecting the current state of the LED. This step demonstrates the integration of web-based control mechanisms with real-time IoT device management, allowing the LED state to be monitored and updated via the web server.

- **Root Endpoint:** The `handleRoot()` function serves the HTML page representing the current status of the LED to clients accessing the web server's root URL.
- **Receive Endpoint:** The `handleReceive()` function processes incoming HTTP POST requests at the /receive endpoint. Based on the received data ("ON" or "OFF"), the function updates the LED state and refreshes the HTML page to display the new status. Invalid data triggers an error response, ensuring only valid commands are processed.

6.2.5. Operational example

Command "ON".

- **ESP32-A:** Reads the temperature from the DHT11 temperature sensor and, if the temperature exceeds the threshold, sends the "ON" message to ESP32-B via Bluetooth.
- **ESP32-B:** Receives the "ON" message and transmits it to ESP32-C via serial communication.
- **ESP32-C:** Forwards the "ON" message to ESP32-D via an HTTP POST request over Wi-Fi.

Table 11
Routing information for data transmission in Fig. 3. Each row is stored in the controller register of the corresponding controller mentioned in the left most column named Controller; the other two columns Source Device and Destination Device are stored in the SourceDevices and DestinationDevices in the respective controller registers.

Controller	Source Device	Destination Device
A	A-DHT11	A-B-C-D-LED1
B	B-A-DHT11	B-C-D-LED1
C	C-B-A-DHT11	C-D-LED1
D	D-C-B-A-DHT11	D-LED1

- **ESP32-D:** Receives the message, turns the LED ON, updates the web server, and the HTML page reflects that the LED is now ON.

Command "OFF".

- **ESP32-A:** Reads the temperature from the DHT11 temperature sensor, and if the temperature is below the threshold, sends the "OFF" message to ESP32-B via Bluetooth.
- **ESP32-B:** Receives the "OFF" message and transmits it to ESP32-C via serial communication.
- **ESP32-C:** Forwards the "OFF" message to ESP32-D via an HTTP POST request over Wi-Fi.
- **ESP32-D:** Receives the message, turns the LED OFF, updates the web server, and the HTML page reflects that the LED is now OFF.

This multi-ESP32 system demonstrates a robust IoT network, utilising a combination of Bluetooth, serial, and Wi-Fi communications to manage temperature-based control of an LED. The integration of sensor data with real-time device control, web-based monitoring, and the seamless transition between different communication protocols underscores the interoperability and scalability of IoT systems in real-world applications.

Table 11 shows how the routing information is stored in the controller registers.

7. Experimental results

To propose an interoperable solution for IoT edge devices, it is generally sufficient to demonstrate that the solution enables seamless communication among heterogeneous devices, thus ensuring interoperability [28,29]. However, in this work, we have extended the evaluation by conducting additional experiments to assess the resource efficiency of the proposed solution. Specifically, we have measured the ROM and RAM usage, and Response Latency of our solution in comparison with three existing open standards. These experiments provide a deeper understanding of the resource requirements and efficiency of our solution, further validating its practicality for IoT edge environments.

We describe our evaluation metrics, competitor solutions, and provide performance analysis and discussion.

7.1. Evaluation metrics

In the literature, a related paper [30] has evaluated the performance of four interoperable open standard solutions OGC PUCK over Bluetooth [31], TinySOS [32], SOS over CoAP [30], and OGC SensorThings API [26]. In order to evaluate each solution, it has used a service prototype (i.e., server), a gateway, and a client. As metrics, it has used (*i*) code storage in read only memory (ROM, EEPROM), (*ii*) main memory (RAM) usage, (*iii*) request length of an operation, (*iv*) response size of an operation, (*v*) and response latency.

Our solution differs from existing solutions in terms of data communication. Existing solutions require making API calls with XML, JSON, or plain text data requests to the cloud and then retrieving data from the cloud. In contrast, our solution involves direct communication between the controller and the device, bypassing the cloud entirely. Consequently, metrics like request size and response size, which are relevant for cloud-based systems, are not applicable to our solution as there is no API call or cloud data transmission. In this implementation, we have considered wired connection only. Our solution eliminates the need for data requests by directly sending signals from the controller to the device. So we can evaluate our implementation with three metrics (*i*) ROM storage, (*ii*) RAM usage, and (*iii*) Response Latency.

7.2. Competitor solutions

We target our solution to work both on class-0 and class-1 edge devices [33]. Class-0 devices such as Arduino Uno R3 have only 32 KB of random access memory (RAM) and 2 KB of code space. Class-1 devices such as ESP32 have about 520 KB of SRAM and 128 KB of code space. Since Class-0 devices like the Arduino Uno R3 do not support Bluetooth or Wi-Fi connections, we utilise Class-1 devices like the ESP32 for network interoperability implementation.

We have run the three available open standard solutions–OGC SOS [34], OGC SensorThings API [35], and EdgeX Foundry [27]. These solutions support edge interoperability, but using cloud access needing comparatively more resources. So they cannot operate on class-0 device. We have used class-1 device ESP32 microcontroller for them also. ESP32 has low-power consumption and integrated Wi-Fi and Bluetooth capabilities. This facilitates seamless connectivity, enabling devices to establish connections with local networks and internet. Note that SOS [34] only supports data sensing operations. On the other hand, OGC SensorThings API [35] and EdgeX Foundry [27] support both data sensing and actuator tasking but in two different parts: Part 1 and Part 2 respectively. However, we cannot run their actuator tasking in Part 2 because of the unavailability of the program and documentations. So our comparison with the three open standards is mainly restricted within the data sensing operations.

The three open standard solutions do not directly support Bluetooth connectivity. To integrate data from Bluetooth devices into these solutions, the data must first be collected, processed, and formatted into a standardised format compatible with the respective standard. Once formatted, the data is typically stored in a cloud-based system, requiring Wi-Fi connectivity to interact with the Application Programming Interface (API). While these standards allow for the insertion and retrieval of data from Bluetooth devices, the underlying communication with the cloud-based system relies on Wi-Fi, making them indirect solutions for Bluetooth integration. Thus, only one microcontroller was used in experiments with the three open standards, whereas our framework implementation used multiple controllers to demonstrate network interoperability between Wi-Fi and Bluetooth.

7.3. Feature comparison

In Table 12, we compare this implementation of InterEdge's Network level interoperability with existing solution's Network level interoperability. The table shows that OGC SOS, OGC SensorThings API, and EdgeX Foundry require cloud access to facilitate edge interoperability. The table also shows that the architectures of OGC SOS, OGC SensorThings API, and EdgeX Foundry do not explicitly address interoperability at distinct levels, such as network interoperability levels, but rather approach the issue in a more general manner. Note that this work explicitly addresses interoperability at different levels. This implementation also has integrated controllers to provide services related to the network interoperability level.

Table 12
Network edge interoperability: our implementation vs existing solutions.

Interoperability framework implementation	Cloud support needed	Integrated controller?	Explicit network level?
OGC SOS [34]	✓	×	×
OGC SensorThings API [35]	✓	×	×
EdgeX Foundry [27]	✓	×	×
This Work	×	✓	✓

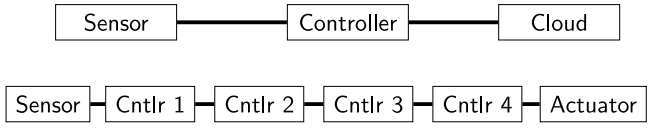


Fig. 4. Test cases: (top) for data sensing for the three open standard solutions, and (bottom) in a multiple controller scenario for network interoperability.

7.4. Test cases

Fig. 4 presents the schematic diagram of the experimental setup used to compare our solution with three open standard solutions.

Fig. 4 (top) illustrates the schematic diagram for the three open standards. To evaluate the performance of our solution in alignment with the three open standards, our analysis focused on the process of reading sensor data.

For the OGC SOS [25], this involved a twofold procedure: the insertion of observations (Insert Observation) into the SOS cloud followed by their retrieval (Get Observation). These functions were assessed separately due to their distinct roles within the OGC SOS framework. Within the OGC SensorThings API [26], evaluations were conducted on both the creation (Create Observation) and retrieval (Read Observation) of observations. Likewise, in EdgeX Foundry, we scrutinised the functionalities associated with sending data (Send Data) and reading data (Read Data).

For convenience, throughout the rest of the paper, we will refer to SOS (Insert Observation) as SOS (Cloud Upload), SOS (Get Observation) as SOS (Cloud Download), SensorThings API (Create Observation) as SensorThings API (Cloud Upload), SensorThings API (Read Observation) as SensorThings API (Cloud Download), EdgeX (Send Data) as EdgeX (Cloud Upload), and EdgeX (Read Data) as EdgeX (Cloud Download). For the three standard solutions, 'writing' refers to reading data from the sensor, sending a request to the cloud to store the data, and receiving a success response (Cloud Upload).

In contrast, for our solution, 'writing' refers to sending a command directly to an actuator. Similarly, for the three standard solutions, 'reading' involves sending a request to the cloud to retrieve previously stored data and successfully reading the data from the cloud (Cloud Download). For our solution, 'reading' refers to directly reading the sensor data.

Fig. 4 (bottom) depicts our network level implementation for both read and write data in the multiple controller scenario. For our network level implementation we have used all physical devices.

7.5. Performance comparison

As mentioned before, we show the performance comparison using ROM usage, RAM usage, and Response Latency.

Table 13
ROM and RAM usage (in KB).

Solution name	ROM	RAM
SOS (Cloud Upload)	1032.805	47.22
SOS (Cloud Download)	1030.049	47.22
SensorThings API (Cloud Upload)	1030.141	47.22
SensorThings API (Cloud Download)	1028.921	46.997
EdgeX (Cloud Upload)	1028.961	47.012
EdgeX (Cloud Download)	1028.777	46.996
InterEdge (Bluetooth) (ESP32-A)	1107.449	39.492
InterEdge (Bluetooth) (ESP32-B)	1109.793	39.492
InterEdge (Wi-Fi) (ESP32-C)	1025.321	46.824
InterEdge (Wi-Fi) (ESP32-D)	937.481	45.188

7.5.1. ROM usage

Table 13 shows the ROM usage by various solutions. The SOS (Cloud Upload) needs the most ROM in comparison with other implementations. This is because it needs to send a big chunk of data for insertion of the sensor data. The SOS (Cloud Download) and SensorThings API (Cloud Upload) take similar sizes of ROM. This is because the size of XML data is used for the SOS (Cloud Download) request and the size of JSON data used for SensorThings API (Cloud Upload) are similar. For SensorThings API (Cloud Download), EdgeX (Cloud Upload), and EdgeX (Cloud Download) use similar sizes of ROM. SensorThings API (Cloud Download) requires a bit less ROM than SOS (Cloud Download) because the size of data SensorThings API (Cloud Download) sends to retrieve sensor data is less than the size of data SOS (Cloud Download) sends to retrieve sensor data.

In this study, the implementation is carried out using physical devices. For this project, it necessitates memory space with the use of libraries for network connectivity, such as Bluetooth and Wi-Fi. Our experiments demonstrate that the Bluetooth library occupies more ROM space compared to the Wi-Fi library.

For the InterEdge (Bluetooth) (ESP32-A), it is necessary to start a Bluetooth server and transmit data over the Bluetooth connection. Similarly, for the InterEdge (Bluetooth) (ESP32-B), the process involves scanning available Bluetooth connections, connecting to the desired device, and receiving data from it.

For the InterEdge (Wi-Fi) (ESP32-C), both the Wi-Fi and HttpClient libraries are required to interact with the web server created by the ESP32-D. Likewise, for the InterEdge (Wi-Fi) (ESP32-D), the WebServer library is essential for setting up the web server.

7.5.2. RAM usage

Table 13 shows the RAM usage during the execution of each implementation. Since all three open standards primarily involve calling URLs with POST or GET requests, their RAM usage is quite similar.

Our experiment on network-level interoperability shows that while the Bluetooth library occupies more ROM, it requires less RAM to establish a connection, resulting in improved performance compared to the three open standards. Conversely, the Wi-Fi library demonstrates RAM usage comparable to the open standards, as they all rely on Wi-Fi connectivity, though it still requires less RAM than the open standards. This experiment indicates that the three open standards demand significantly more RAM than our solution.

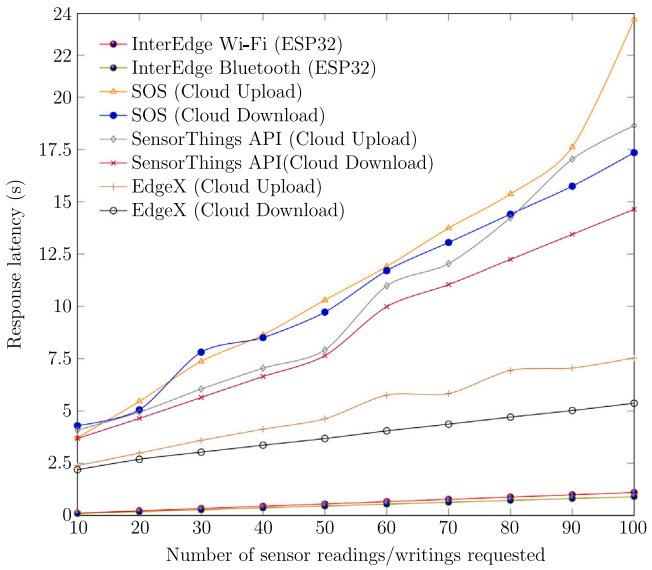


Fig. 5. Response latency vs. number of sensor readings/writings.

7.5.3. Response latency

In our study, we measured the end-to-end response latency. Using a PC client, we retrieved sensor data from ESP32 devices for our network-level interoperability implementation. For the three open standards, latency was defined as the time taken from when the PC client sent a request to the cloud to when it received a response. In our network interoperability setup, latency for Wi-Fi was measured from the moment the controller initiated a web server API call until it received a response. For Bluetooth, latency was calculated from when the controller began reading sensor data to when it transmitted the data to another controller.

Fig. 5 shows the response latency vs the number of readings/writings request for the competitor solutions. We see that the SOS (Cloud Upload) operation exhibited the longest response time. This can be attributed to the greater size of XML data transmitted compared to other implementations. For a lower number of sensor readings or writings, SensorThings API (Cloud Upload) demonstrated shorter response times than SOS (Cloud Download). However, for over 80 sensor readings or writings, SensorThings API (Cloud Upload) took longer. Notably, EdgeX implementations exhibited comparatively shorter response times than SOS and SensorThings API implementations. Specifically, EdgeX (Cloud Download) operation exhibited shorter response times than (Cloud Upload), as it only accessed the cloud once to retrieve all the data at once. Thus, any increase in time was solely due to the additional data retrieved for higher numbers of sensor readings.

In our network interoperability implementation, both Bluetooth and Wi-Fi demonstrated significantly lower response times compared to the three open standards. Although all three open standards utilise Wi-Fi connectivity, their reliance on complex HTTP requests with larger data sizes contributed to their increased response times. In our Wi-Fi scenario, the initial setup of the web server on the controller took 4826 ms, but this setup time was excluded from our response latency measurements, as the three open standards were also measured from the point a request was sent to the cloud until a response was received. Similarly, for Bluetooth, the scanning, searching, and connection establishment process introduces an initial delay, but this was also excluded from our response latency calculations to ensure a fair comparison. Additionally, the three open standards were deployed on a local server (localhost), meaning that if they were hosted on actual cloud servers, their response times would likely be even higher than the recorded values.

7.6. Overall discussion

This paper addresses network-level interoperability, focusing on efficient Bluetooth and Wi-Fi implementations to optimise resource usage and latency. Our Bluetooth network implementation demonstrates notably efficient RAM usage, outperforming the other three standards. For Wi-Fi, our solution also shows slightly reduced RAM consumption compared to the three open standards. However, our approach does require more ROM because it delivers a full-stack, end-to-end interoperability framework. In contrast, the open standards (SOS, SensorThings API, and EdgeX Foundry) rely on their predefined frameworks, simplifying implementation by allowing data exchange through basic HTTP POST or GET requests to their API endpoints.

In our experimental setup, we call URLs directly in the open standards for HTTP requests. In comparison, our solution involves a more comprehensive approach, encompassing the entire stack from device communication to data processing and control actions. When considering just the isolated HTTP request–response cycle of our approach, our solution outperforms the other three open standards.

This demonstrates the effectiveness of our approach in providing an integrated, resource-efficient framework for network-level interoperability across Bluetooth and Wi-Fi, especially in scenarios that require minimal latency for real-time IoT applications. One limitation of this study is that it is primarily designed for small-scale networks with a limited number of devices. This is due to the fact that the research is specifically focused on enabling network edge interoperability to support mission-critical services. So the scalability of the proposed solution depends on each controller's storage capacity to manage routing information as the network expands.

8. Conclusion

Network interoperability in IoT enables devices using different communication protocols, like Wi-Fi and Bluetooth, to connect and exchange data within a single network. This is essential for building scalable and flexible IoT systems, as it allows diverse devices to communicate seamlessly regardless of their native protocols. Achieving this interoperability often involves creating standardisation across protocols, data formats, and communication methods.

This paper addresses the critical challenge of achieving network interoperability at the IoT edge level. We have implemented a lightweight solution and showed seamless data exchange between Bluetooth and Wi-Fi devices. This approach reduces reliance on cloud-based architectures and minimises latency. By prioritising resource efficiency, our approach improves the performance of IoT systems, particularly in resource-constrained environments. Our solution significantly outperforms traditional standards like SOS, SensorThings API, and EdgeX in response latency, ROM and RAM usage. While competitive with EdgeX, our approach is more lightweight and efficient. This work demonstrates the value of edge-level network interoperability solutions for creating adaptable and efficient IoT networks. By facilitating smooth communication between diverse devices, we open the door to more scalable, flexible, and low-latency IoT applications across different network protocols.

CRediT authorship contribution statement

Tanzima Azad: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Conceptualization. **M.A. Hakim Newton:** Writing – review & editing, Supervision, Conceptualization. **Jarrod Trevathan:** Writing – review & editing, Supervision. **Abdul Sattar:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] K. Pitstick, M. Novakouski, G.A. Lewis, I. Ozkaya, Defining a reference architecture for edge systems in highly-uncertain environments, in: 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C), 2024, pp. 356–361, http://dx.doi.org/10.1109/ICSA-C63560.2024.00064.

[2] S. Bojadjievski, N. AnastasovaBojadjievska, M. Kalendar, A. Tentov, Interoperability of emergency and mission critical IoT data services, in: 2018 26th Telecommunications Forum, TELFOR, 2018, pp. 1–4, http://dx.doi.org/10.1109/TELFOR.2018.8611826.

[3] O. Bello, S. Zeadally, M. Badra, Network layer inter-operation of Device-to-Device communication technologies in Internet of Things (IoT), Ad Hoc Netw. 57 (2017) 52–62, http://dx.doi.org/10.1016/j.adhoc.2016.06.010.

[4] Z. Qin, G. Denker, C. Giannelli, P. Bellavista, N. Venkatasubramanian, A Software Defined Networking architecture for the Internet-of-Things, in: 2014 IEEE Network Operations and Management Symposium, NOMS, 2014, pp. 1–9, http://dx.doi.org/10.1109/NOMS.2014.6838365.

[5] J.R. Prasad, S.P. Bendale, R.S. Prasad, Semantic internet of things (IoT) interoperability using software defined network (SDN) and network function virtualization (NFV), in: Semantic IoT: Theory and Applications: Interoperability, Provenance and beyond, Springer International Publishing, 2021, pp. 399–415, http://dx.doi.org/10.1007/978-3-030-64619-6_18.

[6] P. Thubert, Objective Function Zero for the Routing Protocol for Low-Power and Lossy Networks (RPL), in: Request for Comments, (6552) RFC Editor, 2012, http://dx.doi.org/10.17487/RFC6552, RFC 6552.

[7] 6LoWPAN: The wireless embedded internet, in: 6LoWPAN, John Wiley & Sons, Ltd, 2009, p. 244, http://dx.doi.org/10.1002/9780470686218.

[8] T. Azad, M.A.H. Newton, J. Trevathan, A. Sattar, Hierarchical decentralized edge interoperability, IEEE Internet Things J. 11 (8) (2024) 13948–13960, http://dx.doi.org/10.1109/JIOT.2023.3340298.

[9] R. Vilalta, A. Mayoral, D. Pubill, R. Casellas, R. Martínez, J. Serra, C. Verikoukis, R. Muñoz, End-to-end SDN orchestration of IoT services using an SDN/NFV-enabled edge node, in: 2016 Optical Fiber Communications Conference and Exhibition , OFC, 2016, pp. 1–3.

[10] O. Flauzac, C. González, A. Hachani, F. Nolot, SDN based architecture for IoT and improvement of the security, in: 2015 IEEE 29th International Conference on Advanced Information Networking and Applications Workshops, 2015, pp. 688–693, http://dx.doi.org/10.1109/WAINA.2015.110.

[11] K. Tantayakul, R. Dhaou, B. Paillassa, Impact of SDN on mobility management, in: 2016 IEEE 30th International Conference on Advanced Information Networking and Applications, AINA, 2016, pp. 260–265, http://dx.doi.org/10.1109/AINA.2016.57.

[12] R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, R. Boutaba, Network function virtualization: State-of-the-art and research challenges, IEEE Commun. Surv. Tutor. 18 (1) (2016) 236–262, http://dx.doi.org/10.1109/COMST.2015.2477041.

[13] M. Thoma, S. Meyer, K. Sperner, S. Meissner, T. Braun, On IoT-services: Survey, classification and enterprise integration, in: 2012 IEEE International Conference on Green Computing and Communications, 2012, pp. 257–260, http://dx.doi.org/10.1109/GreenCom.2012.47.

[14] H. Tschofenig, J. Arkko, D. Thaler, D.R. McPherson, Architectural Considerations in Smart Object Networking, in: Request for Comments, (7452) RFC Editor, 2015, http://dx.doi.org/10.17487/RFC7452, RFC 7452.

[15] H.M. Zahid, Y. Saleem, F. Hayat, F.Z. Khan, R. Alroobaea, F. Almansour, M. Ahmad, I. Ali, A framework for identification and classification of IoT devices for security analysis in heterogeneous network, Wirel. Commun. Mob. Comput. (1) (2022) 8806184, http://dx.doi.org/10.1155/2022/8806184.

[16] A. Sivanathan, H.H. Gharakheili, F. Loi, A. Radford, C. Wijenayake, A. Vishwanath, V. Sivaraman, Classifying IoT devices in smart environments using network traffic characteristics, IEEE Trans. Mob. Comput. 18 (8) (2019) 1745–1759, http://dx.doi.org/10.1109/TMC.2018.2866249.

[17] D. Chasaki, C. Mansour, Security challenges in the internet of things, Int. J. Space- Based Situated Comput. 5 (3) (2015) 141–149, http://dx.doi.org/10.1504/IJSSC.2015.070945.

[18] Z. Sheng, S. Yang, Y. Yu, A.V. Vasilakos, J.A. Mccann, K.K. Leung, A survey on the ietf protocol suite for the internet of things: standards, challenges, and opportunities, IEEE Wirel. Commun. 20 (6) (2013) 91–98, http://dx.doi.org/10.1109/MWC.2013.6704479.

[19] C.-L. Zhong, Z. Zhu, R.-G. Huang, Study on the IOT architecture and gateway technology, in: 2015 14th International Symposium on Distributed Computing and Applications for Business Engineering and Science, DCABES, 2015, pp. 196–199, http://dx.doi.org/10.1109/DCABES.2015.56.

[20] E. Ferro, F. Potorti, Bluetooth and Wi-Fi wireless protocols: a survey and a comparison, IEEE Wirel. Commun. 12 (1) (2005) 12–26, http://dx.doi.org/10.1109/MWC.2005.1404569.

[21] T. Cooklev, Wireless communication standards: A study of IEEE 802.11, 802.15, 802.16, in: Wireless Communication Standards, John Wiley & Sons, Ltd, 2011, p. 372, http://dx.doi.org/10.1002/9781118098837.

[22] J. Gutierrez, M. Naeve, E. Callaway, M. Bourgeois, V. Mitter, B. Heile, IEEE 802.15.4: a developing standard for low-power low-cost wireless personal area networks, IEEE Netw. 15 (5) (2001) 12–19, http://dx.doi.org/10.1109/65.953229.

[23] E. Perahia, C. Cordeiro, M. Park, L.L. Yang, IEEE 802.11ad: Defining the next generation multi-gbps wi-fi, in: 2010 7th IEEE Consumer Communications and Networking Conference, 2010, pp. 1–5, http://dx.doi.org/10.1109/CCNC.2010.5421713.

[24] E. Khorov, I. Levitsky, I.F. Akyildiz, Current status and directions of IEEE 802.11be, the future Wi-Fi 7, IEEE Access 8 (2020) 88664–88688, http://dx.doi.org/10.1109/ACCESS.2020.2993448.

[25] D.G. Leibovici, R. Santos, G. Hobona, S. Anand, K. Kamau, K. Charvat, B. Schaap, M. Jackson, Geospatial standards, in: The Routledge Handbook of Geospatial Technologies and Society, Taylor & Francis, 2023, http://dx.doi.org/10.4324/9780367855765-7.

[26] S. Liang, T. Khalafbeigi, H. van Der Schaaf, B. Miles, K. Schleidt, S. Grellet, M. Beaufils, M. Alzona, OGC SensorThings API Part 1: Sensing version 1.1, in: Open Geospatial Consortium, 2021.

[27] E. Foundry, Why EdgeX, 2021.

[28] E. Lee, Y.-D. Seo, S.-R. Oh, Y.-G. Kim, A survey on standards for interoperability and security in the internet of things, IEEE Commun. Surv. Tutor. 23 (2) (2021) 1020–1047, http://dx.doi.org/10.1109/COMST.2021.3067354.

[29] S.S. Albouq, A.A.A. Sen, N. Almashf, M. Yamin, A. Alshanqiti, N.M. Bahbouh, A survey of interoperability challenges and solutions for dealing with them in IoT environment, IEEE Access 10 (2022) 36416–36428, http://dx.doi.org/10.1109/ACCESS.2022.3162219.

[30] M.A. Jazayeri, S.H. Liang, C.-Y. Huang, Implementation and evaluation of four interoperable open standards for the internet of things, Sensors 15 (9) (2015) 24343–24373, http://dx.doi.org/10.3390/s150924343.

[31] T. O'Reilly, D. Toma, J. Del-Rio-Fernandez, K. Headley, OGC® PUCK protocol standard, in: Open Geospatial Consortium, OGC, OGC Candidate Encoding Standard, Wayland, MA, USA, 2012.

[32] M.A. Jazayeri, C.-Y. Huang, S.H.L. Liang, TinySOS: design and implementation of interoperable and tiny web service for the internet of things, in: Proceedings of the First ACM SIGSPATIAL Workshop on Sensor Web Enablement, SWE '12, Association for Computing Machinery, 2012, pp. 39–46, http://dx.doi.org/10.1145/2451716.2451722.

[33] C. Bormann, A.P. Castellani, Z. Shelby, CoAP: An application protocol for billions of tiny internet nodes, IEEE Internet Comput. 16 (2) (2012) 62–67, http://dx.doi.org/10.1109/MIC.2012.29.

[34] A. Na, M. Priest, Sensor observation service. Version 1.0, 2007, Open Geospatial Consortium.

[35] S. Liang, T. Khalafbeigi, OGC SensorThings API Part 2–tasking core, version 1.0, 2019, Open Geospatial Consortium.